ZOKA QUANTIC

Toward Post-Quantum Privacy by Default — Version 2.2 (Design Draft, revised)

Status: Design draft, revised on 23 October 2026 against the source code. Zoka has two networks, both started from zero: **Zoka Quantic Testnet**, with test coins, live since its genesis on 23 October 2026, with the official desktop wallet ZSilent 3.0.5 and a CLI miner for Windows and Linux; and **Zoka Quantic Mainnet**, which launches only with the fully post-quantum pool (section 4.8) and an external review. **Zenith**, ZOKA's first network, was the rehearsal that led to Zoka Quantic (section 1); it was retired on 21 October 2026, and nothing from its chain, balances or addresses carries over to either network. The testnet runs the protocol described here and restarts from zero, with new addresses, when the post-quantum pool is ready. This document is a proposal, not a security audit.

Status labels used throughout:

Label	Meaning
Implemented	Present in source and tested; does not imply a public binary for every platform.
In development	Being built.
Live	Running on its public network.
Planned	Committed, not started.
Removed	Shipped before, no longer part of ZOKA.
Retired	Ran before; no longer runs.
Dropped	Planned before, no longer needed.

Abstract

A public ledger is permanent. Anyone can store it today and analyse it with whatever computers exist in fifteen years. For a privacy chain, a future quantum computer is therefore not a future problem: what it will be able to read is being written now.

Zoka Quantic is the second generation of ZOKA and a new chain. Its rule is simple: **the target is post-quantum privacy from the genesis block** — who received a payment, how much, and which note each transaction spent. On the testnet today, the mechanisms that protect validity — that nobody can create coins or spend what is not theirs — still rest on classical cryptography. The mainnet launches only once they do not: with the fully post-quantum pool of section 4.8.

The order is deliberate. A break of privacy reaches backwards over every transaction already on a chain, and nothing can undo it, so privacy had to be post-quantum first. A break of validity can only be exploited once a large quantum computer exists, so it can be closed later — but before the mainnet holds anything of

value.

Zoka Quantic is **not** "quantum-proof" today, and this document never claims it is. Its privacy is designed to resist future quantum attacks; its validity becomes post-quantum with the new pool, which is in development.

The design combines:

1. **Ironwood shielded transactions**: Zcash's corrected Orchard construction, with Halo 2 proofs and no trusted setup, used through the unmodified `orchard` 0.16 crate. A spend publishes a nullifier and a note-commitment-tree root, never the note it consumes. No rings, no key images.
2. **X-Wing note envelopes**: Orchard encrypts each note with an elliptic-curve key agreement that a quantum computer could undo for anyone who knows the recipient's address. Zoka Quantic replaces it with X-Wing, which combines X25519 and ML-KEM-768 (NIST FIPS 203).
3. **Hybrid addresses** carrying an Orchard address and an X-Wing key: about 1,730 characters, shared as a 57-character **short address** that every node can resolve (section 4.4).
4. **Quantum-recoverable notes** (the note format of ZIP 2005).
5. **Hybrid Ed25519 + ML-DSA-65** (NIST FIPS 204) signatures on bootstrap snapshots and on the team's chain checkpoints.
6. **A fully post-quantum pool** in development (section 4.8): hash-based STARK proofs with the spend authorized inside the proof. The testnet restarts with it; the mainnet launches with it.

Consensus is RandomX proof of work (`rx/0`, which XMRig mines unmodified through a Stratum server built into the node) with a 120-second target, ASERT difficulty and chain selection by accumulated work. The token is **ZKA**, capped at 23,000,000, of which 5,000,000 are allocated to the team at genesis (section 8).

---

1. Why a New Chain

Zenith was the rehearsal. ZOKA's first network, Zenith, is where the node, the wallets, CPU mining and the Tor transport were built and run with real users. It also showed what had to change, and the reasons below are what it taught. Zoka Quantic does not patch Zenith: it is a better chain, designed from zero with that experience.

Historical privacy cannot be repaired retroactively. Every output on the Zenith chain was created with classical key agreement. Changing the rules from some block onward would leave every earlier output readable to a future quantum adversary, forever. A new genesis lets every output be born under the new rules.

Zenith published which note each transaction spent. In October 2026 we found that the Zenith private-transaction format carries, in the clear, the commitment and the one-time public key of every note it consumes. An inspection recorded on 11 October 2026 of a 25,204-block local chain snapshot found that its 254 private transactions publish 834 spent notes; only 254 of them sit inside a ring signature, one per transaction; and 559 can be linked directly to the output that created them. Amounts and recipient addresses are not directly published by those fields; off-chain information can still identify participants. The transaction graph is exposed, and no quantum computer is needed to see it. The ring signature does not help, because the real input is published next to the ring. This is a design error of version 1, and section 4.2 is how Zoka Quantic removes it.

New formats are cleaner at genesis. Zoka Quantic changes everything a transaction contains: the curves, the proof system, the note format and the addresses. That is simpler on an empty ledger than as a fork of a live one, and every output is born under the same rules.

Two networks, never mixed. Zoka Quantic Testnet and Zoka Quantic Mainnet each have their own chain id, protocol identifier, gossip topics, ports, peer list and data folder, and a node of one cannot join the other. Zenith was retired on 21 October 2026. Its coins **do not migrate**: there is no balance claim or conversion into either Quantic network, and each one starts with its own genesis allocation.

---

2. Scope and Status

Component	Status
Quantic transactions: Ironwood with X-Wing note envelopes, hybrid addresses, compact blocks for light wallets	Implemented
Short addresses (57 characters) and the nodes' address directory	Implemented
XMRig-compatible proof of work (`rx/0`) and a Stratum server in the node	Implemented; XMRig mined testnet blocks on 23 October 2026
Quantic node: blocks, mempool, mining, P2P relay, API, snapshot sync, run on local multi-node networks	Implemented
ASERT difficulty, chain selection by work, timestamp rules	Implemented
Hybrid Ed25519 + ML-DSA-65 snapshot and checkpoint signatures, required on both networks	Implemented; each network's keys are created before its genesis
Hybrid TLS (X25519MLKEM768) on iroh connections	Implemented
Wallet command line (`zoka quantic-wallet`) and the CLI miner for Windows and Linux	Released (3.0.4)
ZSilent 3.0.5, the official desktop wallet	Released
Light-wallet transport and phone engine (P2P, through Tor)	Implemented; ZKAPriv screens in development
Zoka Quantic Testnet genesis and first seed	Live since 23 October 2026
Fully post-quantum pool (section 4.8)	In development: prototype measured
Testnet restart with the post-quantum pool	Planned
Support for Monero pool software (fetch work, submit blocks)	Planned
Shared mining pool without custody (P2Pool-style)	Planned
Code-signed Windows releases	Planned
Hybrid TLS on the libp2p transport	Planned
External review of ZOKA's changes	Planned, before the mainnet
Zoka Quantic Mainnet genesis	Planned, only with the complete protocol
Multi-party trusted-setup ceremony	Dropped: Halo 2 has no trusted setup
P2P Exchange (ZKA for USDT)	Removed, October 2026

Component	Status
Zenith, the rehearsal network	Retired, 21 October 2026

No date is committed for the mainnet genesis. The post-quantum pool, its run on the testnet, device measurements and the external review are prerequisites.

---

3. Threat Model

Zoka Quantic is designed to defend against:

1. A passive observer of the public chain, including anyone indexing block data.
2. **A future observer with a cryptographically relevant quantum computer** analysing the archived chain ("harvest now, decrypt later"). That observer must not learn who received an output, how much it carried, or which note a transaction spent, even if it knows the recipient's address.
3. A node operator who reads all gossiped traffic, or who serves blocks to a light wallet.
4. An adversary holding the encrypted wallet file without its password.

It does **not** defend against:

1. **Forgery once such a quantum computer exists.** Halo 2 proofs over the Pasta curves, RedPallas signatures and the binding property of value commitments rest on discrete-logarithm assumptions. A large quantum computer could forge proofs and create coins without detection, or spend outputs it does not own. This risk only runs forward in time. The response is a protocol upgrade to post-quantum validity before such machines exist. Recoverable notes (section 4.6) keep a path open for honest owners; the replacement itself is not implemented.
2. A global passive adversary watching both ends of every network link in real time.
3. A user who gives the same address to separate off-chain identities. Each wallet has a single address (section 4.4), so whoever receives it can connect those identities off chain. Payments to it remain unlinkable on chain.
4. A compromised recovery phrase or spending key. There is no recovery backdoor.
5. Local disk compromise combined with knowledge of the wallet password.
6. The holder of the team's checkpoint key choosing which valid chain is canonical (section 7.4). That key cannot make an invalid block valid.

---

4. Cryptographic Stack

4.1 Primitives

Primitive	Use	Against a quantum computer
Halo 2 (inner-product argument over Pallas/Vesta), Ironwood action circuit	Spend validity: note membership, ownership, nullifier, value	No trusted setup. Soundness rests on discrete logarithms and is not post-quantum
Sinsemilla and Poseidon, inside Orchard	Note commitments, commitment tree, nullifiers	Commitments are hiding; binding rests on discrete logarithms (ZIP 2005 strengthens it for recoverable notes)
Value commitments on Pallas	Hidden amounts and the balance check	Hiding with a uniform trapdoor; binding breaks
RedPallas	Spend-authorization and binding signatures	Not post-quantum
X-Wing: X25519 + ML-KEM-768 (FIPS 203), draft revision 10	Note envelopes	Intended to stay secure if either component holds
ChaCha20-Poly1305	Note and outgoing ciphertexts	Symmetric, 256-bit keys
BLAKE2b	Key derivation, transaction ids, digests	Quantum search reduces security margins only
Ed25519 + ML-DSA-65 (FIPS 204)	Snapshot and checkpoint signatures	Both required; ML-DSA is the post-quantum component
Argon2 + AES-256-GCM	Wallet file encryption	Password entropy, parameters and implementation remain critical
RandomX	Proof of work	No claim of immunity to quantum mining advantage

These are properties of primitives, not an end-to-end proof of Quantic's privacy. Ciphertexts, nullifiers, commitments, signatures and public proof inputs must be reviewed together. The quantum security of the complete protocol remains an acceptance requirement, not an established result.

4.2 Ironwood Transactions — Implemented

A Quantic transaction is an Ironwood bundle: a list of **actions**, each of which spends one note and creates one note. A transfer has at least two; padding uses dummy notes of value zero that cannot be told apart from real ones. Per action, the bundle publishes the nullifier of the spent note, the commitment `cmx` of the new note, a randomized spend-authorization key `rk`, a value commitment `cv_net` and the note envelope (section 4.3). Once per bundle it publishes the anchor (a root of the note-commitment tree), the public value balance and one Halo 2 proof.

The proof shows, for every action, that:

1. the spent note is a leaf of the tree under the anchor, or has value zero;
2. the spender knows its keys, bound to `rk`;
3. the nullifier is correctly derived from the note and the owner's nullifier key;
4. the new note's commitment is well formed;
5. `cv_net` commits to the difference between the spent and the created value, both 64-bit.

A binding signature shows that the value commitments add up to the public value balance, which in a transfer is the fee. Spend-authorization signatures, one per action, sign the transaction id (section 6).

Why not a ZOKA circuit. In May 2026 a soundness flaw was reported in Orchard's circuit, present since 2022, that in theory allowed coins to be created without detection. Zcash fixed it and on 28 July 2026 activated Ironwood, a new pool with the corrected circuit. If the most-reviewed circuit in the ecosystem carried a flaw for four years, a circuit written for ZOKA alone is not acceptable. Zoka Quantic uses the `orchard` 0.16 crate without modifying the circuit and changes only what lives outside the proof: note encryption, addresses, transaction format and consensus.

No trusted setup. Halo 2 needs no per-circuit secret parameters: the proving and verifying keys are derived deterministically from the circuit. The multi-party ceremony planned in the 11 October draft is no longer needed.

4.3 X-Wing Note Envelopes — Implemented

Orchard encrypts each note to its recipient with a Diffie-Hellman exchange on Pallas. Anyone who knows an address could compute its incoming viewing key with a quantum computer and read every note sent to it; ZIP 2005 discusses this case. Zoka Quantic keeps Orchard's note plaintext and replaces the key agreement:

```
eseed     = BLAKE2b-512("ZokaQ_KemEncSeed", rseed || rho || cmx)
(kem, ss) = X-Wing.Encaps(recipient X-Wing key, eseed)            1,120 bytes
key       = BLAKE2b-256("ZokaQ_NoteEncKey", ss || kem || cmx)
enc       = ChaCha20-Poly1305(key, nonce 0, plaintext)            580 bytes
plaintext = 0x03 || diversifier (11) || value (8) || rseed (32) || memo (512)
ock       = BLAKE2b-256("ZokaQ_OutCiphKey", ovk || cv_net || cmx || kem)
out       = ChaCha20-Poly1305(ock, nonce 0, Orchard address || ss)   91 bytes
```

- The recipient accepts a note only if the decrypted fields rebuild the published `cmx` and re-encapsulating with that note's `eseed` gives exactly the published ciphertext: the equivalent of Orchard's check of the ephemeral key.
- The encapsulation randomness comes from the note's secret seed, unique per note. ZOKA's X-Wing implementation is tested against the official X-Wing test vectors.
- Dummy outputs are encapsulated to a throwaway X-Wing key and cannot be told apart from real ones.
- `out` lets the sender recover what they sent with their outgoing viewing key. Without that key, it reveals nothing.
- **Public outputs** use an all-zero outgoing viewing key, so anyone can open `out` and check who received how much. Only the genesis allocation uses one (section 8). Spending it still requires the owner's keys, and the spend cannot be traced.
- The hybrid's shared secret is intended to remain secure if either X25519 or ML-KEM-768 holds. The X-Wing specification is a work-in-progress IETF draft; ZOKA pins revision 10 and its test vectors.

4.4 Keys and Addresses — Implemented

Every key derives from the wallet's recovery phrase (24 words recommended):

```
seed       = BLAKE2b-512("ZokaQuanticSeed_", root)
sk         = Orchard spending key at m/32'/coin_type'/account'   (ZIP 32)
xwing_seed = BLAKE2b-256("ZokaQ_XWingSeed_", sk)
```

`coin_type` is `0x5a4b` on mainnet and 1 on test networks, so the same phrase gives unrelated wallets on each. An address is:

```
content  = version 0x01 || Orchard address (43 bytes) || X-Wing key (1,216 bytes)
checksum = first 4 bytes of BLAKE2b-256("ZokaQ_AddrCheck_", prefix || content)
address  = prefix || Base58(F4Jumble(content || checksum))
```

- About 1,730 characters: too long to share. What wallets show and share is a **short address**, a fingerprint of the full one:

`digest = BLAKE2b-256("ZokaQ_ShortAddr_", prefix || content) short = prefix || Base58(0x02 || digest || checksum) 57 characters on the testnet`

It fits a standard QR code (33 modules with medium error correction) that any phone camera reads. A short address carries no key: to pay it, a wallet looks up the full address in a directory that every node keeps, and checks it against the digest before paying. Wallets publish their full address there on their own, with a small Argon2 proof of work (8 MiB, 8 leading zero bits) against spam; nodes relay records by gossip, at most 30 per peer per minute, and announce their own again every 6 hours. The directory is kept by the nodes, not by consensus, and the full address keeps working everywhere. - **F4Jumble** (ZIP 316) makes every character depend on every byte, so nobody can keep the visible beginning of a real address and swap in their own X-Wing key. - The checksum covers the network prefix: a testnet address does not decode as a mainnet address. - **One address per wallet.** The X-Wing key travels in the address, so several addresses sharing it would be linkable. Change goes to Orchard's internal address, which is never shown. - Prefixes: `zq1` (mainnet), `zqtest1` (testnet) and `zqregtest1` (local test networks). - The exportable viewing key (network, Orchard full viewing key and X-Wing seed: 129 bytes) reveals a wallet's whole history and must be kept secret.

4.5 Light Wallets — Implemented; app screens pending

A phone does not download full blocks. For each action it receives the compact part (nullifier, `cmx`, the X-Wing ciphertext and the first 52 bytes of `enc`: 1,236 bytes) and, per transaction, two digests of the rest. With that it:

1. recomputes every transaction id and the block's transaction-id root, and compares it with the block header it already checked by proof of work;
2. finds its notes by decrypting the compact part (a memo requires the full transaction);
3. detects its own spends by nullifier, and keeps the commitment tree and its spend witnesses itself.

A node cannot hide or alter an output in what it serves without the mismatch showing, and it does not learn which notes belong to the phone: every wallet asks for the same blocks. Phones will ask any connected peer over the P2P network, through Tor, rather than a server.

4.6 Quantum Recoverability — Implemented (note format)

Notes use Ironwood's recoverable plaintext format (lead byte `0x03`), proposed in ZIP 2005. In it, the commitment randomness is derived by hashing every field of the note together with its seed, which the proposal argues makes note commitments binding even against post-quantum adversaries. If discrete-logarithm-based validity ever has to be disabled, a separate recovery protocol could let owners spend these notes by proving that derivation with a post-quantum proof. That protocol does not exist yet, in Zcash or in ZOKA. Using the format from genesis means no Quantic note would need to be re-created for it. For ZOKA the answer is the fully post-quantum pool (section 4.8): the testnet restarts with it and the mainnet launches with it, so no mainnet note ever depends on this path.

4.7 Hybrid Signatures — Implemented; required

Bootstrap snapshots and the team's chain checkpoints (section 7.4) carry an Ed25519 and an ML-DSA-65 signature over the same domain-separated message, and a verifier requires both. Once a network's binary has a post-quantum checkpoint key compiled in, its nodes accept only hybrid checkpoints; a Quantic network without the ML-DSA half has no checkpoint authority at all. Signatures authenticate the publisher; they do not replace block validation. Each network's keys are created by the team before its genesis, and only their public halves are compiled in.

4.8 The Fully Post-Quantum Pool — In development

Ironwood's spend authorization (RedPallas), its proofs (Halo 2 on the Pasta curves) and its value commitments rest on discrete logarithms. With a large quantum computer someone could forge spends or create coins that nobody would notice (section 5). The new pool removes every elliptic curve from a spend:

- **Proofs:** a STARK with FRI, whose security rests on hash functions. The prototype uses Plonky3 over the 31-bit KoalaBear field and its degree-4 extension, with Poseidon2 in the trace, in FRI's commitment tree and in the transcript, and FRI with zero-knowledge hiding.
- **Notes and tree:** commitments and the note tree with Poseidon2; nullifiers as a keyed Poseidon2 PRF.
- **Authorization inside the proof:** the proof shows knowledge of the key that owns each spent note and is bound to the transaction id, so no re-randomizable signature is needed.
- **Balance inside the proof:** one proof covers every input and output of a transaction, with the fee public; no homomorphic commitments.
- **Envelopes:** X-Wing, as today.

First measurements on a PC, one thread (permutations only, without the constraints that tie them into a transfer, so a lower bound): with hiding a trace needs at least about 256 rows; at that size a proof is about 200 KB with about 100 bits of proven security, takes 0.2 s to prove and 13 ms to verify. At about 200 KB per transfer the proofs of a block have to be aggregated into one (recursion) or blocks must grow and nodes drop old proofs after validating them. Plonky3's recursion has no published release or audit yet; that is the main open risk. The phone measurement and the transfer circuit come next.

The testnet restarts from zero with this pool, with new addresses. The mainnet launches only with it.

---

5. What a Quantum Computer Could Do

An adversary with a large quantum computer, reading the archived chain, could…	Zenith	Zoka Quantic
Test whether outputs belong to an address it knows	Yes	Not unless ML-KEM-768 is also broken
Read the amounts and memos of outputs to an address it knows	Yes	Not unless ML-KEM-768 is also broken
Link a spend to the note it consumes	Yes — already exposed without one	No: deriving a nullifier needs a key that is not in the address
Forge proofs or create coins, once the machine exists	Yes	On the testnet today, yes. With the post-quantum pool (section 4.8), not unless its hash functions break
Spend others' outputs, once the machine exists	Yes	Same as the row above

The Zoka Quantic column states the design's hypotheses for the external review, not established results. ZIP 2005 states that Orchard's privacy properties would be retained against post-quantum adversaries without knowledge of the notes' addresses; the X-Wing envelope is ZOKA's answer for adversaries who know them. Zenith's hybrid outputs, released with version 2.1.2, test the same envelope idea, but Zenith's spends still expose the transaction graph.

The version 1.0 whitepaper stated that past privacy would survive a quantum break. For Zenith that is not true, and this table replaces it.

---

6. Transaction Format and Validation — Implemented

Field	Bytes
Version, network, kind (transfer or coinbase), height	4 + 1 + 1 + 4
Number of actions (1 to 64)	2
Per action: `cv_net`, nullifier, `rk`, `cmx`	4 × 32
Per action: envelope (`kem`, `enc`, `out`)	1,120 + 580 + 91
Flags, value balance, anchor	1 + 8 + 32
Halo 2 proof (2,720 + 2,272 per action)	4 + 7,264 with two actions
Spend-authorization signatures and binding signature	64 per action + 64

A two-action transfer is 11,351 bytes, and each extra action adds 4,255. The height field is the block height in a coinbase and the expiry height in a transfer (zero means none). Decoding is strict: exact lengths, canonical encodings, nothing left over. The network is inside the transaction, so what is signed for one

network is invalid on another.

Transaction id. As in ZIP 244, the id is a tree of BLAKE2b digests over the transaction's effects (header; compact action data; memos; the rest of each action; flags, value balance and anchor). It excludes the proof and the signatures, which go into a separate authorization digest. Signatures sign the id, so re-randomizing a signature does not change it. Every block commits to the roots of both.

Every node checks, at mempool admission and again in every block (a miner can include transactions that never entered a local mempool):

1. strict decoding, network and version;
2. shape: flags, a value balance that is not negative (in a transfer it is the fee; nothing else can leave the shielded pool), at most 64 actions, not expired;
3. that the anchor is the note-commitment-tree root at the end of one of the last 100 blocks of the same branch;
4. that no nullifier was spent before or repeats within the block;
5. the Halo 2 proofs and every signature of the block, verified in one batch.

Coinbase. The first transaction of every block, with spends disabled and the empty-tree anchor. It mints exactly the block subsidy plus the block's fees, paid to private notes; the minted amount is public. Its notes enter the commitment tree 60 blocks later, so no anchor contains them, and they cannot be spent, before they mature.

Fees are public: they are the value balance of a transfer. Wallets pay a conventional fee of 0.0001 ZKA per action, with a minimum of two actions (in the style of ZIP 317), so the fee identifies nobody. Nodes require it to relay a transaction; it is not a consensus rule.

Limits: 64 actions per transaction and 512 per block, excluding the coinbase. Miners currently fill at most 96 actions per block, so blocks stay within the gossip size limit.

---

7. Consensus and Network

7.1 Proof of Work and Difficulty — Implemented

ZOKA uses standard RandomX (`rx/0`, the algorithm Monero uses), designed for ordinary CPUs. On Zoka Quantic the hashing blob is 76 bytes, laid out like Monero's, so XMRig and other `rx/0` miners work unmodified:

```
blob  = 0x02 || timestamp (6) || where (32) || nonce (4) || what (32) || 0x00
where = SHA-256("zoka-quantic-pow-v2-where", chain id, previous hash, height)
what  = SHA-256("zoka-quantic-pow-v2-what", Merkle root, state root, extranonce)
```

The state root covers the transaction-id root, the authorization root and the note-commitment-tree root, so the work covers the whole block body. The 64-bit header nonce is `extranonce << 32 | nonce`: a miner changes the 32-bit nonce, and each Stratum connection gets its own extranonce. The hash is compared with the target as a little-endian number, as in Monero, and the RandomX key changes every 2,048 blocks.

Stratum. The node includes a Stratum server (`--stratum host:port`) that speaks the Monero pool protocol. Each miner logs in with its own address, full or short, and every block it finds pays it in full: this is solo mining through a node, not a shared pool. Shares are 16 times easier than a block, and the node recomputes every hash. On 23 October 2026 XMRig 6.25 and 6.26 mined testnet blocks through it, with every share matching the node's hash.

- **Target block time:** 120 seconds.
- **Difficulty: ASERT** (`aserti3-2d`, the algorithm Bitcoin Cash adopted in 2020), anchored at genesis, with a one-hour half-life. If blocks arrive faster than the target, the difficulty doubles for every hour the chain runs ahead of schedule; if nobody mines, it falls on its own. Targets are 256-bit numbers stored in Bitcoin's compact format, with a floor of 12 leading zero bits. The testnet starts at 14 bits. Because the anchor is the genesis timestamp, a chain that starts mining long after its genesis was created stays at the floor until it catches up with the schedule, as the testnet did on its first day; the mainnet genesis will be created at launch.
- **Chain selection by accumulated work.** Each block counts 2^256 / (target + 1) for the target it had to meet, so the selected chain is the one with the most expected computation.
- **Timestamps** must be later than the median of the previous 11 blocks and no more than two hours ahead of the receiving node's clock.
- Finality is probabilistic. There is no validator set and no stake; see section 7.4 for the team's checkpoints.

The half-life and the initial difficulty will be reviewed on the public testnet before mainnet. Zenith's difficulty and chain-selection rules are not carried over.

7.2 Transport

Nodes talk over libp2p. There is no central server: the desktop wallet runs its own Tor and can publish an onion seed, so nodes are reachable without open ports, and the CLI miner reaches the network out through Tor. A mining node waits until it is connected to at least one peer, so that its blocks reach the network. A fresh node can pull a snapshot over iroh, a QUIC transport with hole punching, from any peer that has one; those connections negotiate X25519MLKEM768, so recorded traffic stays sealed even against a future quantum computer, while a classical peer still connects. Each Quantic network uses its own protocol identifiers, gossip topics and peer list, so the two never mix. The libp2p transport still uses Noise with X25519, and Tor is not post-quantum yet.

The Android wallet joins the same mesh through an embedded Tor client and scans on the phone (section 4.5). Public seed servers will be added for faster first contact, in addition to Tor and never instead of it. The network is designed to keep working if every one of them disappears. The libp2p stack has been upgraded to its current major version (0.57); after the update, `cargo audit` reported no known vulnerabilities in the node's dependencies (October 2026), and old and new nodes interoperate.

7.3 Trusted Setup — None

Halo 2 has no trusted setup (section 4.2). Zenith's Groth16 parameters were generated by a single operator, which means that operator could in principle forge proofs on Zenith. That concern does not carry over to Zoka Quantic.

7.4 Team Checkpoints — Implemented; declared trust

The team's node signs checkpoints: "at height H, the canonical block is this hash". Nodes carry the team's public key compiled in and do not follow any branch that contradicts the latest signed checkpoint, even one with more work. This prevents forks below the checkpoint caused by old or faulty clients, a network partition, or someone with more hash power trying to rewrite history.

This is explicit trust in the team, declared here like the team allocation. The key decides which valid chain is canonical. It cannot make an invalid block valid, create coins outside the emission schedule or spend anyone's funds: every node still validates every block. If the key were lost, the network would continue by accumulated work. If it leaked, an attacker could hold the network on a valid branch of their own until a new binary replaces the key. Each Zoka Quantic network has its own key, created by the team before its genesis and signing in the hybrid format (section 4.7).

---

8. Token Economics — ZKA on Zoka Quantic

Parameter	Value
Symbol	ZKA
Decimals	6 (1 atom = 0.000001 ZKA)
Maximum supply	23,000,000 ZKA
Team allocation at genesis	5,000,000 ZKA (21.7%), spendable from the first block after genesis
Mined supply	18,000,000 ZKA
Initial block reward	12 ZKA
Subsidy reduction	After 1,051,200 mined blocks (about 4 years at 120 s)
Coinbase maturity	60 blocks
Tail emission	None

Emission schedule:

Era	Reward	Blocks	Mined in era	Total supply at end of era
—	Genesis	—	—	5,000,000 (team allocation)
0	12 ZKA	1,051,200	12,614,400 ZKA	17,614,400 ZKA
1	6 ZKA	897,600	5,385,600 ZKA	23,000,000 ZKA

The genesis block pays the team allocation and no subsidy. The next 1,051,200 blocks pay 12 ZKA each, the following 897,600 pay 6 ZKA, and the subsidy is zero afterward, about 7.4 years after genesis at the

target block time. Mining can continue for transaction fees; new coin issuance ends. The node numbers the genesis as block 1, so the subsidy runs from block 2 through block 1,948,801. These rules are implemented for Quantic and are not active on Zenith.

On the team allocation. The version 1.0 whitepaper said "no premine, no founder allocation", and Zenith had none. Zoka Quantic has one, and it is stated here plainly: 5,000,000 ZKA to the team, paid in the genesis block as a public output (section 4.3) to an address published in the network parameters. Anyone can open that output and check its recipient and amount against the announcement, and consensus rejects a genesis that pays anything else. The allocation enters the note-commitment tree in the genesis block, so it is spendable from the next block, with no lock-up and no maturity delay. Spending it is as private as any other spend.

Block rewards are paid to private notes. The subsidy amount is public, so emission can be audited; the recipient is not.

---

9. Privacy Objectives — Zoka Quantic

9.1 What the Chain Hides

Field	Mechanism
Recipient	X-Wing note envelope (section 4.3); outputs carry no address
Amount	Value commitments; values are checked inside the proof
Memo	Encrypted inside the envelope
Which note a transaction spends	Membership proven inside the Halo 2 proof (section 4.2)
Sender	No address, key or note of the sender is published

9.2 What the Chain Necessarily Publishes

Field	Reason
Block height, timestamp, proof of work	Consensus
Transaction id; that a transaction occurred; its number of actions	Inherent to a public ledger
Expiry height of a transfer, if set	Lets a transfer lapse instead of waiting forever
Nullifiers	Double-spend prevention; unlinkable to notes without the owner's keys
Note-commitment-tree roots (anchors)	Anchors for the membership proof
Note commitments, randomized keys, value commitments and envelopes	Needed for validation and for recipients; hiding by construction
The fee	Value balance of the transfer; conventional per action
Block subsidy and the genesis allocation	So the supply can be audited

---

10. Honest Limits

- **Protocol privacy is a hypothesis, not a result.** Ironwood is reviewed and in production on Zcash, but ZOKA's changes — the X-Wing envelopes, the address format, the transaction format and the consensus rules — are new and not yet externally reviewed.
- **On the testnet today, validity is classical.** A large quantum computer could forge proofs, create coins or spend others' outputs there. The fully post-quantum pool (section 4.8) is in development; the testnet restarts with it, and the mainnet launches only with it.
- **The post-quantum libraries are young.** The pure-Rust ML-KEM, ML-DSA and X-Wing crates ZOKA uses are pre-1.0 and, to our knowledge, not independently audited. X-Wing is still an IETF draft.
- **Everything is bigger.** A two-action transfer is about 11 KB today, and full addresses are about 1,730 characters (wallets share 57-character short addresses instead). With the post-quantum pool a proof is about 200 KB before aggregation (section 4.8).
- **Short addresses depend on the directory.** A payer's node has to have received the record; until then, only the full address works. A published record shows, in every node's directory, that the address exists and since when. It shows no balance or payment.
- **Mining is solo for now.** The Stratum server pays each block to whoever found it. A pool that pays small miners often does not exist yet.
- **Releases are not code-signed yet.** Windows may warn about them; the published SHA-256 checksums are how a download is verified.
- **Phones pay in time.** On the 32-bit Android test phone, building the proving key takes about 40 seconds when the app starts, and proving a two-action transfer about 20 seconds, with 111 MB of memory. Checking whether an output belongs to the wallet costs about 4 ms of X-Wing decapsulation. Full-scan measurements will be published after the testnet benchmarks.
- **Fees and transaction sizes are public.** The conventional fee keeps fees uniform, but the number of actions shows.

- **The team holds a checkpoint key** (section 7.4) **and 21.7% of the supply**, spendable from the start.
- **Zenith's history stays linkable.** Its transaction graph is public (section 1). Moving to Zoka Quantic does not change what Zenith already published.
- **A global passive adversary** watching both ends of every link can correlate timing. Tor reduces this risk; it does not remove it.
- **There is no recovery.** A lost seed is lost funds, and nobody can reverse a transaction.

---

11. Roadmap

Done:

1. October 2026 security update: desktop Tor and node dependencies updated.
2. Removal of the P2P Exchange.
3. Post-quantum building blocks (ML-KEM-768, X-Wing, ML-DSA-65) and on-device benchmarks.
4. Ironwood chosen over a ZOKA-specific circuit, which removed the trusted setup.
5. Quantic transactions, addresses and light-wallet data, as a library.
6. The Quantic node, run on local multi-node networks, including chain reorganizations.
7. Hybrid snapshot and checkpoint signatures, required on both networks.
8. ASERT difficulty and chain selection by work.
9. The light-wallet transport for phones and the phone engine, tested end to end against a real node.
10. Networking-stack upgrade (libp2p 0.57) and hybrid TLS on iroh connections.
11. ZSilent 3.0, for Zoka Quantic only, tested against a real node.
12. Zenith retired after serving as the rehearsal; Zoka has two networks, both from zero.
13. Zoka Quantic Testnet genesis (23 October 2026), with the team's node as the first seed over Tor.
14. Short addresses and a standard QR code.
15. XMRig-compatible proof of work and a Stratum server in the node.
16. ZSilent 3.0.5, the official desktop wallet, and the CLI miner for Windows and Linux.

Next, in order, without dates:

1. Quantic screens in ZKAPriv.
2. Support for Monero pool software, so existing pools can add Zoka.
3. The fully post-quantum pool: transfer circuit, per-block aggregation, phone measurements.
4. Testnet restart from zero with the pool.
5. Hybrid TLS on the libp2p transport; public seed servers, in addition to Tor; code-signed releases.
6. External review.
7. Zoka Quantic Mainnet genesis.
8. After the mainnet: a shared mining pool without custody, in the style of P2Pool.

---

12. Reference Code

Source repositories are hosted on GitHub at github.com/zoka-network and GitLab at gitlab.com/zokanetwork. Projects include the node (`zokanetwork`, which also holds the CLI miner in `packaging/cli-miner`), the desktop wallet (`zsilent-core`), the Android wallet (`zkapriv`), the web explorer of Zenith, the rehearsal network (`zokaexplorer`), and this site (`zokaweb`). The transaction library is `crates/zoka-quantic`; the node is built with the `quantic` feature, without which it cannot join either network. The post-quantum pool prototype is `crates/zoka-pq-pool`. Releases are verified with their published SHA-256 checksums.

Primary references:

- NIST FIPS 203: ML-KEM.
- NIST FIPS 204: ML-DSA.
- X-Wing, revision 10 (work in progress).
- The Halo 2 book.
- ZIP 224: Orchard Shielded Protocol.
- ZIP 2005: Ironwood Quantum Recoverability (proposed).
- ZIP 244: Transaction Identifier Non-Malleability.
- ZIP 316: Unified Addresses (F4Jumble).
- ZIP 317: Proportional Transfer Fee Mechanism.
- ASERT difficulty adjustment (aserti3-2d).

These references describe primitives and constructions; none audits ZOKA.

---

13. Closing Position

ZOKA exists because the right to transact in private is the last frontier of individual freedom. Version 1.0 promised the strongest privacy that implementation knew how to ship, with its limits in plain text. Version 2 keeps that promise and adds what we learned since: privacy has to be measured against the computers of the future, and against our own mistakes. Both are written down here.

Verify what is true. Reveal nothing else.

— Natosh Zoka, core dev @ ZokaNetwork